객체와 클래스 스터디: 거북이 보물찾기 게임

참고: 한선관, 김태령, “Chapter 13. 객체와 클래스 프로젝트”, 컴퓨팅 사고를 위한 파이썬, 생능출판

게임 시나리오

- 입력

  1. 초록 거북(주인공)의 이동속도를 입력 받는다.
  2. 적 거북의 숫자를 입력 받는다.
  3. 적 거북의 이동 속도를 입력 받는다.
  4. 보물(사각형)의 개수를 입력 받는다.

- 처리

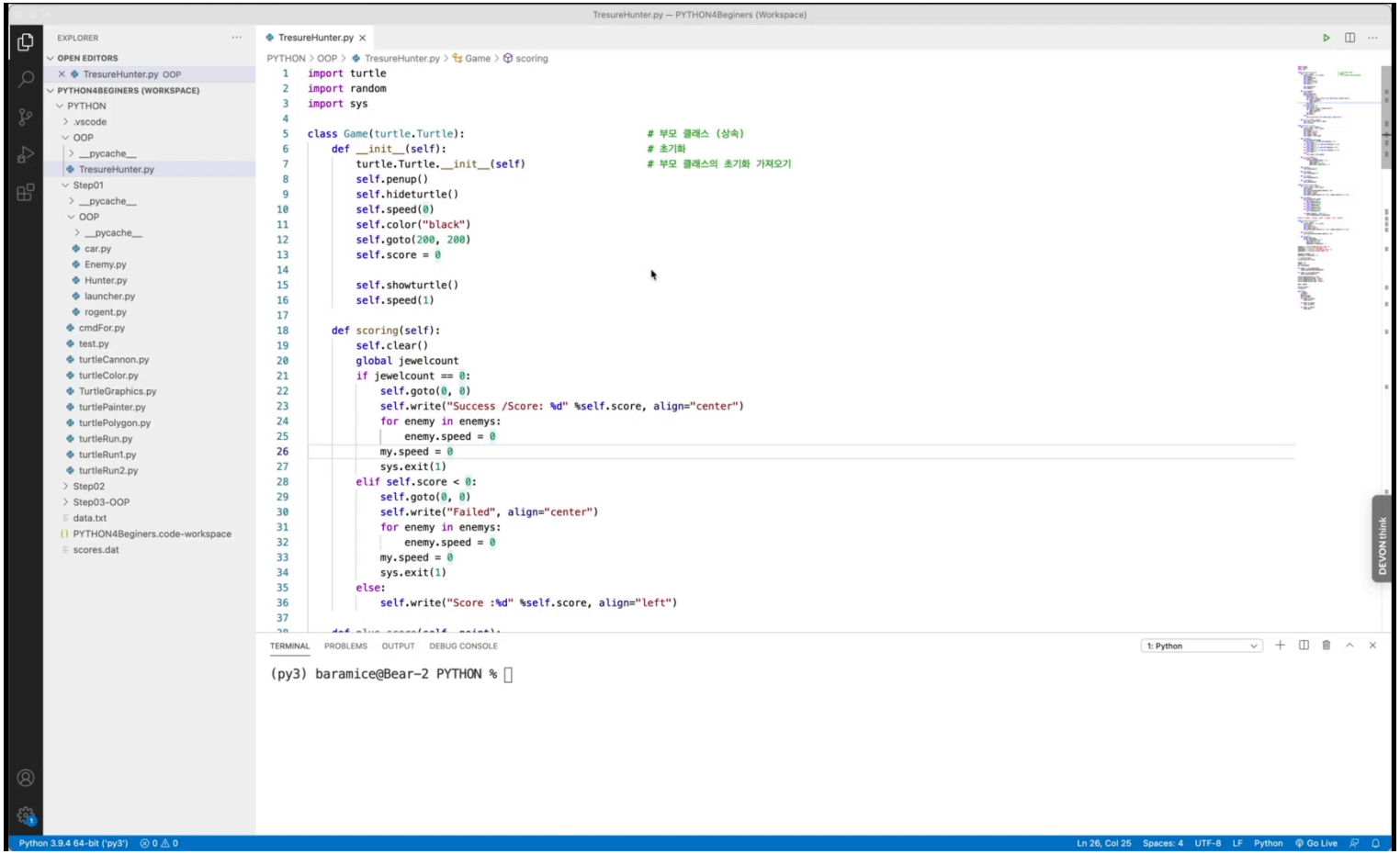
  1. 초록 거북이 돌아다니면서 보물을 얻는다.
  2. 초록 거북이 보물을 얻으면 10점씩 증가한다.
  3. 적 거북은 무작위로 돌아다니면서 초록 거북을 위협한다.
  4. 적 거북과 초록 거북이 충돌하면 10점씩 감소한다.

- 출력

  1. 보물을 다 얻으면 화면에 Success 라고 출력하고 종료한다.
  2. 만약 점수가 0점보다 낮아지면 Failed 라고 출력하고 종료한다.

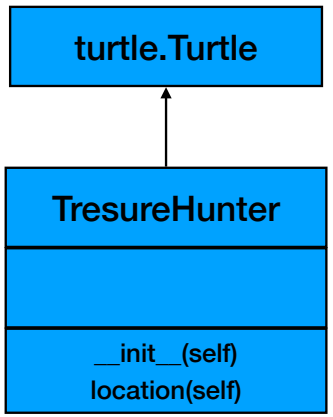
게임 시나리오

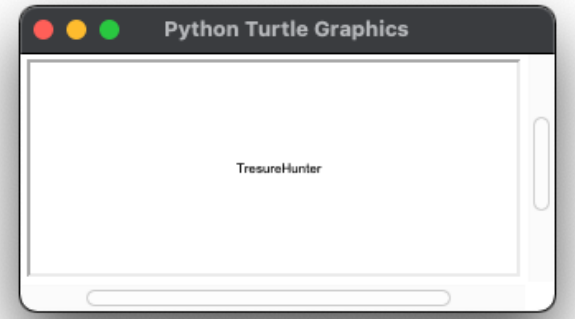
- 실행화면


TresureHunter.py — PYTHON4Beginners (Workspace)
EXPLORER
OPEN EDITORS
TresureHunter.py OOP
PYTHON4BEGINERS (WORKSPACE)
PYTHON
.vscode
OOP
__pycache__
TresureHunter.py
Step01
__pycache__
OOP
__pycache__
car.py
Enemy.py
Hunter.py
launcher.py
rogent.py
cmdFor.py
test.py
turtleCannon.py
turtleColor.py
TurtleGraphics.py
turtlePainter.py
turtlePolygon.py
turtleRun.py
turtleRun1.py
turtleRun2.py
Step02
Step03-OOP
data.txt
PYTHON4Beginners.code-workspace
scores.dat
TresureHunter.py
PYTHON > OOP > TresureHunter.py > Game > scoring
import turtle
import random
import sys
class Game(turtle.Turtle): # 부모 클래스 (상속)
    def __init__(self): # 초기화
        turtle.Turtle.__init__(self) # 부모 클래스의 초기화 가져오기
        self.penup()
        self.hideturtle()
        self.speed(0)
        self.color("black")
        self.goto(200, 200)
        self.score = 0
        self.showturtle()
        self.speed(1)
    def scoring(self):
        self.clear()
        global jewelcount
        if jewelcount == 0:
            self.goto(0, 0)
            self.write("Success /Score: %d" %self.score, align="center")
            for enemy in enemys:
                enemy.speed = 0
            my.speed = 0
            sys.exit(1)
        elif self.score < 0:
            self.goto(0, 0)
            self.write("Failed", align="center")
            for enemy in enemys:
                enemy.speed = 0
            my.speed = 0
            sys.exit(1)
        else:
            self.write("Score :%d" %self.score, align="left")
TERMINAL
PROBLEMS
OUTPUT
DEBUG CONSOLE
1: Python
(py3) baramice@Bear-2 PYTHON %
DEVONthink
Python 3.9.4 64-bit ('py3')
Ln 26, Col 25
Spaces: 4
UTF-8
LF
Python
Go Live

클래스 정의와 객체 생성

```
import turtle

class TresureHunter(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.hideturtle()
        self.goto(0,0)
        self.write("TresureHunter", align="center")

    def location(self):
        h = self.heading()
        x = self.xcor()
        y = self.ycor()
        dis = self.distance(self)

        print("Location of TresureHunter: ")
        print("X Coor:", x, ", Y Coor:", y, ", Direction:", h)
        print("Distance:", dis)

a = TresureHunter()
a.location()

turtle.mainloop()
```





클래스 정의와 객체 생성

- 여러 객체 생성

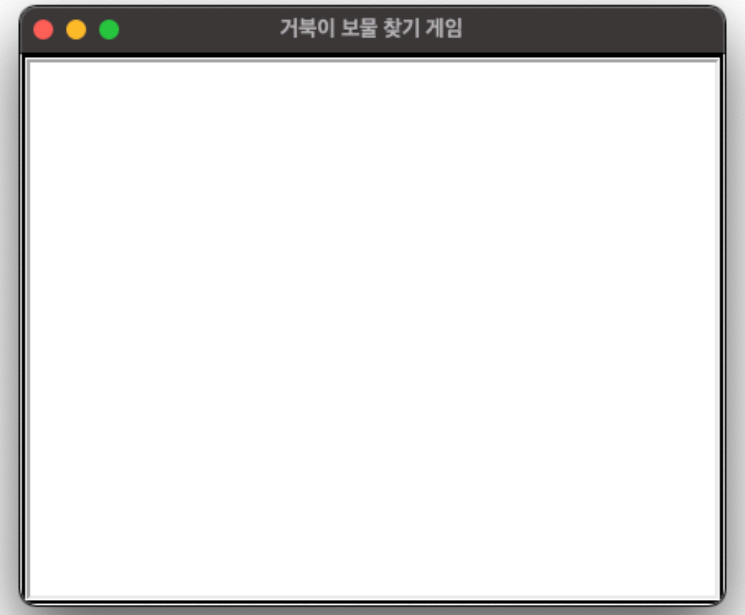
```
import turtle

class TresureHunter(turtle.Turtle):                    # 부모 클래스 (상속)
    def __init__(self):                                # 초기화
        turtle.Turtle.__init__(self)                   # 부모 클래스의 초기화 가져오기
        self.penup()
        self.goto(0,0)

        self.showturtle()
        self.speed(1)

    def location(self):""
```

```
turtles = []                                   # list()
for a in range(5):                             # 5번 반복
    turtles.append(TresureHunter())            # TresureHunter 거북이를 리스트에 객체 넣기

for a in turtles:                              # 객체 순회 interator
    a.fd(100)                                  # 하나씩 100 전진
```

게임 루프

- turtle.Screen()

```
import turtle

s = turtle.Screen()
s.title("거북이 보물 찾기 게임")

s.tracer(0)                          # 화면을 빠르게 업데이트
turtle.listen()

while True:                          # 게임 메인 루프
    s.update()
```



보물 찾는 거북이(TresureHunter)

- Hunter 클래스
  - 주인공 클래스(my)
  - 펜 올리기, 거북 모양, 초록색
  - 키보드 상하좌우 화살표

```python
import turtle

class Hunter(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.color("green")
        self.speed = speed

    def up(self):
        self.setheading(90)

    def down(self):
        self.setheading(270)

    def left(self):
        self.setheading(180)

    def right(self):
        self.setheading(0)

    def move(self):
        self.forward(self.speed)
```

보물 찾는 거북이(TresureHunter)

- Hunter 클래스
- 게임 루프

```python
import turtle

class Hunter(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.color("green")
        self.speed = speed

    def up(self):
        self.setheading(90)

    def down(self):
        self.setheading(270)

    def left(self):
        self.setheading(180)

    def right(self):
        self.setheading(0)

    def move(self):
        self.forward(self.speed)
```

```python
import turtle

s = turtle.Screen()
s.title("거북이 보물 찾기 게임")

my = Hunter(0.1)

turtle.onkeypress(my.up, “Up”)
turtle.onkeypress(my.down, “Down”)
turtle.onkeypress(my.left, “Left”)
turtle.onkeypress(my.right, “Right”)

s.tracer(0)
turtle.listen()

while True:
    s.update()
    my.move()
```

보물을 지키는 거북이(Enemy)

- Enemy 클래스
  - 적 클래스(enemy)
  - 다양한 색의 거북이, 첫 위치 랜덤
  - 지정된 속도만큼 앞으로 이동

```
import turtle
import random

colors = ["beige", "yellow", "pink", "orange", "red", "black"]  # 적 거북 컬러 선택지

class Enemy(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.speed = speed
        self.color(colors[random.randint(0, 5)])                # 다양한 색
        self.goto(random.randint(-250, 250), random.randint(-250, 250))

    def move(self):
        self.forward(self.speed)
```

보물을 지키는 거북이

- Hunter 클래스

```
colors = ["beige", "yellow", "pink", "orange", "red", "black"]

class Enemy(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.speed = speed
        self.color(colors[random.randint(0, 5)])
        self.goto(random.randint(-250, 250), random.randint(-250, 250))


    def move(self):
        self.forward(self.speed)
        # 0.1% 확률로 주인공을 따라온다.
        if random.randint(1, 1000) == 1:
            self.setheading(self.toward())
```

- 게임 루프

```
enemy(0.2)

while True:                          # 게임 메인 루프
    enemy.move()
```

보물을 지키는 거북이

- 벽 충돌 방지하기 Enemy.move()
  - x 좌표가 250을 넘어가면 거북이 머리를 반대 방향으로 돌린다. 각 네 방향에 대하여 똑같이 적용한다.

```
    def move(self):
        self.forward(self.speed)
        if self.xcor() > 250:
            self.setheading(180)
        elif self.xcor() < -250:
            self.setheading(0)
        elif self.xcor() > 250:
            self.setheading(270)
        elif self.ycor() < -250:
            self.setheading(90)

        if random.randint(1, 1000) == 1:
            self.setheading(self.towards(my))
```

보물을 지키는 거북이

- 벽 충돌 방지하기 Enemy.move()
  - 좌표를 넘어가면서도 해당 방향을 계속 보고 있으면 속도 0, 다른 방향이면 회복

```
    def move(self):
        self.forward(self.speed)
        if self.xcor() > 250 and self.heading() == 0:
            self.speed = 0
        elif self.xcor() < -250 and self.heading() == 180:
            self.speed = 0
        elif self.ycor() > 250 and self.heading() == 90:
            self.speed = 0
        elif self.ycor() < -250 and self.heading() == 270:
            self.speed = 0
        else:
            self.speed = self.speed2
```

보물(보석)

- Jewel 클래스
  - 네모난 보석
  - 여러 색을 이용하여 보석 색이 계속 바뀜
  - 보물 찾는 거북이 매우 가까이 다가오면, 보석이 화면 바깥으로 이동

```
class Jewel(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.color("red")
        self.shape("square")
        self.goto(random.randint(-250, 250), random.randint(-250, 250))

    # 반짝이게 만들기
    def twinkle(self):
        self.color(colors[random.randint(0, 5)])    # colors 리스트 이용

jewel = Jewel()

while True:                          # 게임 메인 루프
    jewel.twinkle()
```

게임 만들기

- 점수, 게임 진행 클래스
  - 거북이를 생성해 해당 위치에 배치한다.
  - 점수를 오른쪽 상단에 기재한다.
    - 만약 점수가 0보다 작아면 모든 거북들이 멈추고 난 후 화면 중간에서 Failed를 출력
    - 만약 점수가 10점이 되면 화면 가운데 Success를 출력
  - 점수 변수 및 +10 더하는 메서드

게임 만들기

- 점수, 게임 진행 클래스
- 게임 루프

```
class Game(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.hideturtle()
        self.speed(0)
        self.color("black")
        self.goto(200, 200)
        self.score = 0

    def scoring(self):
        self.clear()
        if jewelcount == 0:
            self.goto(0, 0)
            self.write("Success /Score: %d" %self.score, align="center")
            enemy.speed = 0
            my.speed = 0
        elif self.score < 0:
            self.goto(0, 0)
            self.write("Failed", align="center")
            enemy.speed = 0
            my.speed = 0
        else:
            self.write("Score :%d" %self.score, align="left")

    def plus_score(self, point):
        self.score = self.score + point

game = Game()               # 객체 생성

while True:                 # 게임 메인 루프
    game.scoring()
```

적 거북이 / 보물 충돌 이벤트

- 점수 기능
  - 상승 조건: 보석에 닿으면 10점이 증가한다. 닿은 보석은 사라진다.
  - 하락 조건: 적 거북에 닿으면 10점이 감소한다. 닿은 적 거북은 사라진다.

```
# Jewel 클래스
    def eat(self):

# Hunter 클래스
    def collision(self):

# 게임 메인 루프
while True:
    my.collision()
    jewel.eat()
```

[과제] 게임 기능 확장하기

- 게임 종료

  sys 모듈

  게임 종료 시 sys.exit(1)로 게임 루프 종료

- 스크린 배경 그림 넣기

```
s=turtle.Screen()
s.bgpic("배경 그림 파일")
```

- 캐릭터, 적 거북, 보석 이미지 바꾸기

  image = "이미지 파일"

  s = turtle.Screen()

  s.addShape(image)

  turtle.shape(image)

- 효과음 넣기(따로 재생)

```
import os
os.system('file_path/file_name.mp3')
```